Bilan de la journée — Gastronome Créatif

Mardi 13 octobre 2026 | Projet ASP.NET Core MVC | Recherche et filtres

Aujourd'hui, nous avons avancé sur la recherche dynamique, la communication entre le contrôleur, PostgreSQL et les vues Razor, ainsi que sur le débogage. La recherche textuelle et l'affichage dynamique des catégories fonctionnent ; le filtrage par catégorie reste à finaliser.

1. Développement de la page de recherche

Nous avons travaillé dans Views/Recherche/Index.cshtml et RechercheController.cs afin de permettre aux visiteurs de rechercher des recettes selon un mot-clé et une catégorie.

- Récupération du terme via le paramètre string? terme.
- Interrogation de PostgreSQL avec Entity Framework Core.
- Recherche insensible à la casse grâce à EF.Functions.ILike().
- Affichage des résultats dans Razor et gestion des recherches sans résultat.
- Révision de .AsQueryable(), .Where() et .ToList().

2. Création du filtre dynamique par catégorie

Le menu déroulant contenait initialement des options écrites en dur. Nous l'avons remplacé par une liste générée à partir des catégories de PostgreSQL.

```
var categories = _context.Categories.ToList();
ViewBag.Categories = categories;
```

```
<select id="categorie" name="categorie">
    <option value="">Toutes les catégories</option>
    @foreach (Categories categorie in ViewBag.Categories)
    {
        <option value="@categorie.id_categorie">@categorie.nom</option>
    }
</select>
```

Notions étudiées : collection et élément parcouru dans un foreach, interpolation Razor avec @, différence entre la valeur transmise (value) et le texte visible, correspondance entre name="categorie" et le paramètre du contrôleur, association du label à son champ avec for et id.

Résultat validé : les catégories de PostgreSQL apparaissent dans le menu déroulant.

3. Résolution de l'erreur HTTP 500

La page provoquait une System.NullReferenceException dans Views/Recherche/Index.cshtml, ligne 58. L'analyse du contrôleur a montré cette instruction incorrecte :

```
ViewBag.Categories = categorie;
```

La variable categorie était un identifiant facultatif (int?), et non la liste. La liste était bien récupérée, mais pas transmise à Razor. Nous avons corrigé l'affectation :

```
ViewBag.Categories = categories;
```

Résultat : l'erreur 500 a disparu. Leçon : récupérer des données de la base et les transmettre à une vue sont deux opérations distinctes.

4. Amélioration des conditions de recherche

Nous avons adapté la détection d'une recherche pour prendre en compte un mot-clé ou une catégorie :

```
ViewBag.RechercheEffectuee =
    !string.IsNullOrWhiteSpace(terme) || categorie != null;
```

L'opérateur logique || (OU) convient car un seul critère suffit pour lancer une recherche. Nous avons également corrigé le filtre textuel pour ne l'appliquer que si un terme est saisi :

```
if (!string.IsNullOrWhiteSpace(terme))
```

Les deux filtres .Where() peuvent se cumuler : rechercher « chocolat » dans « Dessert » doit retourner uniquement les recettes répondant aux deux critères.

5. Tests fonctionnels et débogage

Scénario	Résultat
« chocolat » sans catégorie	Fonctionne
« Dessert » sans mot-clé	À corriger
« chocolat » + « Dessert »	À corriger

Pour diagnostiquer le problème, nous avons ajouté temporairement :

```
Console.WriteLine($"Catégorie reçue : {categorie}");
```

Le terminal a affiché « Catégorie reçue : » sans valeur. Cela indique que le paramètre categorie arrivait à null dans les tests. Nous avons donc ciblé la structure du formulaire HTML : le select pourrait se trouver hors du form. Cette hypothèse doit encore être vérifiée.

Un avertissement EF Core relatif à Take() sans OrderBy() a aussi été observé. Il est indépendant du dysfonctionnement du filtre.

6. Point de reprise — mercredi 14 octobre 2026

- Examiner les balises <form> et </form> dans Views/Recherche/Index.cshtml.
- Vérifier que <select name="categorie"> appartient au formulaire envoyé.
- Retester la valeur reçue par RechercheController.
- Valider les trois scénarios de recherche.
- Retirer le Console.WriteLine() de débogage une fois le problème résolu.

Méthode de travail conservée : avancer une étape à la fois, comprendre chaque correction et écrire soi-même le code.

7. Compétences travaillées

Architecture MVC ; transmission des données contrôleur–vue ; requêtes LINQ et Entity Framework Core ; HTML dynamique avec Razor ; formulaires GET ; model binding ; lecture des journaux et débogage ; conditions logiques et composition de filtres.

Bilan général

Acquis : recherche textuelle opérationnelle, catégories alimentées dynamiquement par PostgreSQL et erreur 500 résolue. À terminer : transmission de la catégorie sélectionnée au contrôleur pour activer le filtrage par catégorie et la recherche combinée.